



Advanced Terraform on AWS Lab7

Change Governance

Contents

Expected Duration	1
Prerequisites	1
Phase 1. Classifying Change	2
Phase 2. Change Authorisation	11
Phase 3. Governance-of-Governance	17

Expected Duration

Phase 1 of this lab is expected to take between #-# minutes

Phase 2 of this lab is expected to take #-# minutes

Phase 3 of this lab is expected to take #-# minutes

Total expected lab time is therefore #-# minutes

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Prerequisites

This lab assumes the existence of a remote state backend, as created in Lab 4. Run Phase 2 of Lab 4 now if you do not have remote state backend configured.

This lab also assumes that Lab6a and Lab6b have been completed successfully.

Phase 1. Classifying Change

Overview

In the previous lab you performed a real environment-scoped promotion using GitHub Actions. Everything worked. The promotion was deliberate, approved, controlled and auditable in execution.

And yet, the most important question remained unanswered:

Where was the decision to change an environment explicitly declared?

In this lab, you will not add more Terraform complexity. Instead, you will redesign the promotion flow so that automation is no longer allowed to infer intent. Configuration change intent must be explicit, deliberate, durable, and unambiguous. Automation must refuse to act unless that intent exists.

Starting State

This lab assumes that AWS Lab 6b has been completed successfully. At this point:

- Environment-scoped GitHub Actions workflows are in place
- pr-plan-dev, pr-plan-test, and pr-plan-prod workflows exist
- change-dev, change-test, and change-prod workflows exist
- change-all has been removed from env-roots
- GitHub Environments and approvals exist for test and prod
- GitHub OIDC authentication to AWS is working
- Terraform state is stored remotely in S3 using native lock files
- Network module v1.0.3 has been promoted to DEV
- Network module v1.0.2 remains current in TEST and PROD
- Compute module remains pinned at v1.0.0 in DEV, TEST and PROD

You are not fixing a broken system. You are fixing an incomplete decision model.

If you catch yourself thinking “we could just infer this from the pull request, the approval or the version pin,” pause. That is exactly the behaviour this lab removes.

Objective

By the end of this lab, you should be able to say, with evidence:

Environment-level-only changes are classified as such. Module-level changes are classified as promotions and happen because they have been explicitly declared, with automation refusing to run until that declaration exists.

Defining Change Intent

The blast radius of changing ordinary environment settings is usually smaller than changing which reusable module version an environment consumes. A module version pin change can alter shared infrastructure behaviour and therefore deserves stronger evidence of intent.

The four change classifications introduced in this lab are:

Change Type	What the Human Did	What the Pipeline Concludes
ENV_ROOT_CHANGE	Changed environment configuration only. No module version pins changed.	Local environment modification.
PROMOTION_INTENT	Changed a module pinned version and declared matching intent.	Explicit promotion.
IMPLICIT_PROMOTION_ATTEMPT	Changed a module pinned version without declaring matching promotion intent.	Tried to promote shared code by stealth.
NO-OP_PROMOTION_INTENT	Declared promotion intent without changing a module version pin.	Recorded intent with no effective action.

You will now introduce promotion intent declaration files and update the existing change workflows to enforce intent validation.

The promotion declaration file exists in version control, is human-authored, is environment-specific and carries no executable logic. The updated workflow reads the declaration and checks it against the Terraform module pins before allowing execution to continue.

Introduce Promotion Intent Artifacts

Move the intent tfvars artifacts into the env-roots repository.

1. Copy the supplied promotion intent files from the lab artifacts into the matching environment folders:

From c:\qatfadvaws\artifacts\tfvars\promotion\{env}-promotion.tfvars

To c:\pipeline-repos\env-roots\envs\{env}\{env}-promotion.tfvars

2. From **env-roots**, create a branch, add the files and push:

```
cd c:\pipeline-repos\env-roots
```

```
git checkout main
```

```
git pull
```

```
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D  
$_Trim() }  
git checkout -b intent-files-upload
```

```
git add envs\dev\dev-promotion.tfvars  
git add envs\test\test-promotion.tfvars  
git add envs\prod\prod-promotion.tfvars
```

```
git commit -m "Introduce intent declaration files"  
git push -u origin intent-files-upload
```

3. Create a pull request from **intent-files-upload** to **main**.
4. Observe the relevant PR plan workflows.
5. Merge the pull request once the plan workflows complete.
6. Allow the triggered change workflows to run as no-ops, granting permissions and approvals as required.

Enable Intent Governance

The change workflows contain a FEATURE_PHASE setting. At the end of Lab 6B this was set to 1, meaning Terraform only. You will now enable **intent validation** by changing it to 2 in each change workflow.

7. Locally, update these three files in env-roots/.github/workflows:

```
change-dev.yml  
change-test.yml  
change-prod.yml  
Change FEATURE_PHASE: "1" to "2"
```

8. Commit and push the workflow changes:

```
cd c:\pipeline-repos\env-roots  
git checkout main  
git pull  
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_Trim() }  
git checkout -b intent-gate  
  
git add .github\workflows\change-dev.yml  
git add .github\workflows\change-test.yml  
git add .github\workflows\change-prod.yml
```

```
git commit -m "Introduce intent gate"
git push -u origin intent-gate
```

9. Create a pull request from intent-gate to main.

10. Review and merge the pull request.

Release Compute Module v1.0.1

You will now create a new compute module version. This is deliberately a small tag-only style change. The content of the change is not the point; the promotion governance is the point.

11. Open the compute module repository:

```
c:\module-repos\terraform-aws-module-compute
```

12. Open locals.tf and update the manager value from:

Michael Coulling-Green (CompMod v1.0.0)

to

Oliver Walsh (CompMod v1.0.1)

13. Commit, tag, and push the module release:

```
cd c:\module-repos\terraform-aws-module-compute
git status
git add locals.tf
git commit -m "Publish compute module v1.0.1"
git tag v1.0.1
git push origin main
git push origin v1.0.1
```

14. In GitHub, open the **terraform-aws-module-compute** repository and confirm that the v1.0.1 tag exists.

At this point, the module version exists, but no environment consumes it yet.

Promotion Attempt 1: PROMOTION_INTENT

You will now perform a promotion, as you did in Lab 6B, but this time with intent declared before execution. You will promote the **compute** module v1.0.1 to PROD.

Expected result: SUCCESS - PROMOTION_INTENT

15. Edit **envs/prod/prod-promotion.tfvars** and declare the intended module refs:

```
promotion_reason = "Compute update"
```

```
approved_refs = {  
  network = "v1.0.2"  
  compute = "v1.0.1"
```

16. Edit **envs/prod/main.tf** and set the pinned compute module version to **v1.0.1**

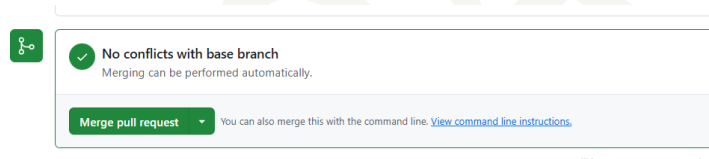
17. Push the changes:

```
cd c:\pipeline-repos\env-roots  
git checkout main  
git pull  
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }  
git checkout -b promote-prod-1  
git add envs\prod\prod-promotion.tfvars  
git add envs\prod\main.tf  
git commit -m "Declare intent and promote compute v1.0.1 to PROD"  
git push -u origin promote-prod-1
```

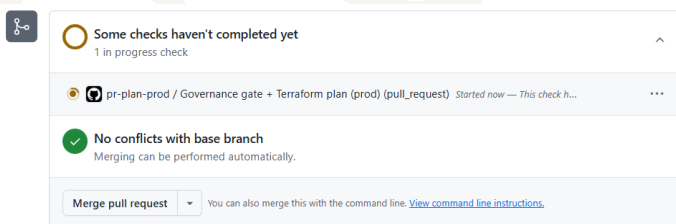
18. Create a pull request from **promote-prod-1** to **main**.

19. Observe **pr-plan-prod**. At this stage intent is visible but apply-time intent governance is still the main enforcement point.

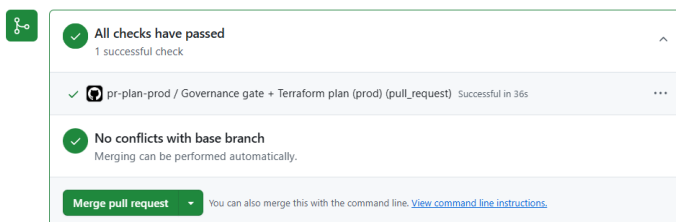
Note: The PR may not initially display the plan workflow



so wait for the plan workflow to initialize ...



and complete....



20. Merge the pull request after **pr-plan-prod** completes.
21. Approve the **change-prod** workflow when prompted.
22. Observe that **change-prod** reads the declared intent and validates it before Terraform execution...

```
496 === Promotion intent validation ===
497 Declared environment      : prod
498 Approved module refs:
499   - compute = v1.0.1
500   - network = v1.0.2
501 Pinned ref (compute) : v1.0.1
502 Pinned ref (network) : v1.0.2
503 Promotion intent validated successfully.
504
505 === Terraform execution ===
506 Change type (final)      : PROMOTION_INTENT
507
```

Pause and ask: Where is the promotion decision now recorded? Is it inferred, or explicitly stated?

Promotion Attempt 2: **IMPLICIT_PROMOTION_ATTEMPT**

Now attempt a promotion without explicit intent declaration. This proves that changing configuration alone is not enough. Promotion must also be declared explicitly in the intent file.

Expected result: FAILURE - IMPLICIT_PROMOTION_ATTEMPT

23. Edit **envs/prod/main.tf** and change the pinned compute module version from v1.0.1 back to **v1.0.0**

24. Push the change:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b promote-prod-2
git add envs\prod\prod-promotion.tfvars
git add envs\prod\main.tf
git commit -m "Promote compute v1.0.0 to PROD without intent"
git push -u origin promote-prod-2
```

25. Create the pull request.
26. Allow the PR plan to run.
27. Merge the pull request and approve the triggered **change-prod** workflow.

Expected observation: change-prod detects that a module pin changed without matching intent and stops before Terraform execution.

```

496 Error: IMPLICIT PROMOTION ATTEMPT
497
498 Environment : prod
499 A module ref was changed (e.g. ?ref=...) but no corresponding update was made to the promotion intent file.
500 Promotion must be explicitly declared: pinned ref change + promotion.tfvars change (same commit).

```

28. Because the code change merged to main before the governance failure, roll back the merge:

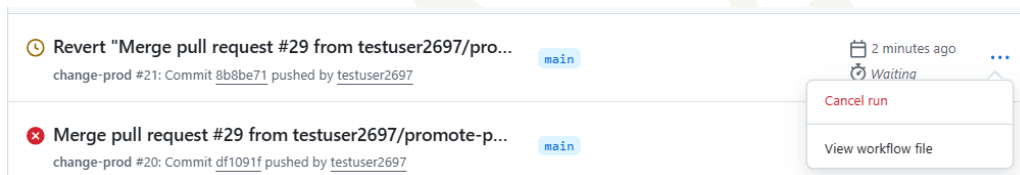
```

cd c:\pipeline-repos\env-roots
git checkout main
git pull
git revert -m 1 $(git rev-list --merges -n 1 HEAD) --no-edit
git push

```

29. The rollback commit restores the files in Git, but it will trigger another change-prod workflow run. Because the rollback modifies only the promotion intent file, the workflow will classify the commit as an IMPLICIT_PROMOTION_ATTEMPT and fail. This behaviour is expected and does not indicate that the rollback was unsuccessful. The repository has already been returned to a valid state.

30. If desired, the workflow run may be cancelled once it has started, as the rollback has already restored the repository to its previous state. Allowing the workflow to continue simply demonstrates how the governance engine classifies the rollback commit.



This triggers a no-op governance run. You can let it complete or cancel it once the rollback is safely committed.

Promotion Attempt 3: NO-OP_PROMOTION_INTENT

Now prove that declared promotion intent must also be accompanied by the appropriate configuration change.

Expected result: FAILURE - NO-OP_PROMOTION_INTENT

31. Edit **envs/prod/prod-promotion.tfvars** and declare compute **v1.0.0** as the approved ref, but **do not** change envs/prod/main.tf

32. Push the intent-only change:

```

cd c:\pipeline-repos\env-roots
git checkout main
git pull

```

```
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b promote-prod-3
git add envs\prod\prod-promotion.tfvars
git commit -m "Declare compute rollback intent without pin change"
git push -u origin promote-prod-3
```

33. Create the pull request and allow planning to complete....

34. Merge the pull request and approve **change-prod**.

35. Expected observation: the change workflow detects declared promotion intent with no matching module version pin change and **stops** by design.

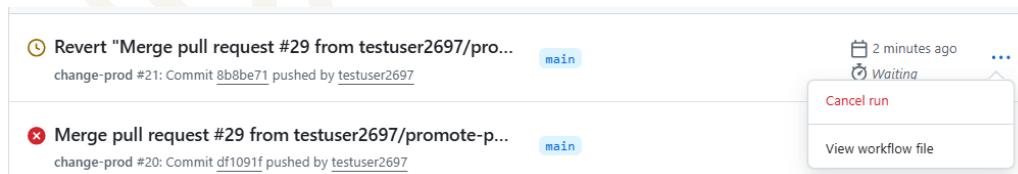
```
496 Error: NO-OP PROMOTION INTENT
497
498 Environment : prod
499 The promotion intent file was updated, but no pinned module ref change was detected.
500 This would record a 'promotion intent' without an actual promotion.
```

36. Roll back the merge:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git revert -m 1 $(git rev-list --merges -n 1 HEAD) --no-edit
git push
```

37. The rollback commit restores the files in Git, but it will trigger another change-prod workflow run. Because the rollback modifies only the promotion intent file, the workflow will classify the commit as a NO-OP_PROMOTION_INTENT and fail. This behaviour is expected and does not indicate that the rollback was unsuccessful. The repository has already been returned to a valid state.

38. If desired, the workflow run may be cancelled once it has started, as the rollback has already restored the repository to its previous state. Allowing the workflow to continue simply demonstrates how the governance engine classifies the rollback commit.



Non-Promotion Update Attempt 1: ENV_ROOT_CHANGE

Now prove that a configuration change that does not include a module promotion is classified as an environment-level change.

Expected result: SUCCESS - ENV_ROOT_CHANGE

39. Edit **envs/prod/terraform.tfvars**. Change a safe environment-level setting, such as a subnet CIDR value, tag value, or similarly harmless lab variable. Change the data subnet CIDR to...

"10.52.99.0/24"

40. Do not edit envs/prod/main.tf.

41. Do not edit envs/prod/prod-promotion.tfvars.

42. Push the change:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b change-prod-env-1
git add envs\prod\terraform.tfvars
git commit -m "Non-module environment update"
git push -u origin change-prod-env-1
```

43. Create the pull request.

44. Allow pr-plan-prod to run.

45. Merge the pull request.

46. Approve the triggered change-prod workflow.

47. Expected observation: the workflow classifies the change as ENV_ROOT_CHANGE because no module version pins changed. Intent declaration is not required for this lower-risk class of change...

```
496  === Terraform execution ===
497  Change type (final)   : ENV_ROOT_CHANGE
```

Reflection

Aspect	Lab 6B	Lab 7
Promotion control	Strong	Strong
Approval gates	Yes	Yes
Execution safety	Yes	Yes
Explicit intent artifact	No	Yes
Audit clarity	Weak	Strong

- Promotion is now declared before execution

- Automation cannot act without intent
- The change decision exists as a durable artifact
- Approval now authorises execution of a declared decision
- Audit questions can be answered cleanly

This is the difference between “we ran the pipeline” and “we decided to promote production.”

In real enterprises, failures rarely come from broken automation. They often come from inferred decisions, implicit intent, and missing declarations.

You have now designed a system where humans decide, automation executes, intent is explicit and evidence is durable.

Lab 6B proved that you could safely change an environment. Lab 7 now proves why you did.

Phase 2. Change Authorisation

You have redesigned the change workflow so that automation can no longer infer intent. Changes now require a durable, explicit, human-authored declaration. However, a critical governance question remains:

Governance Model

A change will now be permitted only when intent is explicitly declared, **and** the declaration is made by an authorised owner. Correct intent from the wrong person must fail.

This part introduces a second independent contract:

- Promotion Intent answers: What change is being proposed?
- Environment Authority answers: Who is allowed to make the change?

Change Type	What the Human Did	Pipeline Conclusion	Authorisation
ENV_ROOT_CHANGE	Changed environment config only	Local environment modification	Allowed with authority
PROMOTION_INTENT	Changed pinned module version and declared matching intent	Explicit promotion	Allowed with authority
IMPLICIT_PROMOTION_ATTEMPT	Changed a module pin without declaring intent	Tried to promote shared code by stealth	Always blocked; authority is irrelevant

NO-OP_PROMOTION_INTENT	Declared intent without changing a module pin	Recorded intent with no effective action	Always blocked; authority is irrelevant
------------------------	---	--	---

Identity Discussion: GitHub Usernames

In this AWS/GitHub Actions environment, ownership is based on the GitHub username exposed to workflows as github.actor.

This keeps the lab simple and reliable because GitHub always provides github.actor to the workflow. In production, organisations may choose a stronger or more centralised identity source, such as GitHub Teams, CODEOWNERS, SSO groups, or an external change-management platform.

For this lab, the rule is simple: the GitHub user that triggered the workflow must be listed as an owner of the target environment.

Enforce Authorisation in the Pipeline

1. Update c:\qatfadvaws\artifacts\governance\owners.yml to include yourself as an owner of DEV. but not TEST or PROD. This allows for unauthorised-promotion testing.
2. Create a top-level **governance** folder in env-roots and copy the modified **owners.yml** file into it:

From: **c:\qatfadvaws\artifacts\governance\owners.yml**

To: **c:\pipeline-repos\env-roots\governance\owners.yml**

3. Update the three change workflows to enable ownership logic by changing **FEATURE_PHASE** from 2 to **3**:

.github\workflows\change-dev.yml

.github\workflows\change-test.yml

.github\workflows\change-prod.yml

4. Commit and push the governance enforcement changes:

```
cd c:\pipeline-repos\env-roots
```

```
git checkout main
```

```
git pull
```

```
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
```

```
git checkout -b ownership-gates
```

```
git add governance\owners.yml
```

```
git add .github\workflows\change-dev.yml
```

```
git add .github\workflows\change-test.yml
git add .github\workflows\change-prod.yml
git commit -m "Introduce ownership enforcement gates"
git push -u origin ownership-gates
```

5. Create a pull request from **ownership-gates** to **main**.
6. Review and merge the pull request.

Test 1 - Attempt an Unauthorised Promotion

You will now attempt a valid promotion intent from an unauthorised actor.

Expected result: FAILURE - unauthorised actor

7. In **envs/prod/prod-promotion.tfvars**, declare compute **v1.0.0** as the approved compute ref.
8. In **envs/prod/main.tf**, set the pinned compute module version to **v1.0.0**
9. Push the changes:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b promote-compute-v100-prod
git add envs\prod\prod-promotion.tfvars
git add envs\prod\main.tf
git commit -m "Promote compute v1.0.0 to PROD"
git push -u origin promote-compute-v100-prod
```

10. Create the pull request.
11. Allow **pr-plan-prod** to run.
12. Merge the pull request.
13. Approve the triggered **change-prod** workflow.
14. Confirm that the workflow fails before Terraform execution.

The failure message should identify the GitHub actor; the authorised owners and the reason execution was blocked.

Although the workflow failed, the commit reached main. Roll it back:

```
git checkout main
git pull
git revert -m 1 $(git rev-list --merges -n 1 HEAD) --no-edit
```

git push

The rollback commit restores the files in Git, but it will trigger another `change-prod` workflow run. If desired, the workflow run may be cancelled once it has started, as the rollback has already restored the repository to its previous state.

Test 2 - Perform an Authorised Promotion

Now add yourself as an authorised prod owner and repeat the promotion.

15. Update **governance/owners.yml** so the prod owners list includes your GitHub username.

16. Commit and merge the ownership update:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b update-prod-ownership
git add governance\owners.yml
git commit -m "Update PROD ownership"
git push -u origin update-prod-ownership
```

17. Create and merge the pull request.

18. Reset **prod-promotion.tfvars** and **main.tf** so both specify compute **v1.0.0**

19. Push the authorised promotion:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b promote-compute-v100-prod-authorised
git add envs\prod\prod-promotion.tfvars
git add envs\prod\main.tf
git commit -m "Authorised compute promotion to PROD"
git push -u origin promote-compute-v100-prod-authorised
```

20. Create the pull request.

21. Allow **pr-plan-prod** to run.

22. Merge the pull request.

23. Approve **change-prod**.

24. In the AWS Console, confirm the relevant PROD compute resource tags have reverted to the v1.0.0 manager value.

Expected result: ownership validation passes, intent validation passes and Terraform execution proceeds normally.

Optional Test Matrix

#	Owner?	Change	Expected Result
1	N	Non-module environment config change	ENV_ROOT_CHANGE Ownership relevant
2	N	Update module refs without matching intent	IMPLICIT_PROMOTION_ATTEMPT Ownership irrelevant
3	N	Update intent file without matching module refs	NO-OP_PROMOTION_INTENT Ownership irrelevant
4	N	Update intent and matching module refs	PROMOTION_INTENT Ownership relevant
5	Y	Non-module environment config change	ENV_ROOT_CHANGE
6	Y	Update module refs without matching intent	IMPLICIT_PROMOTION_ATTEMPT Ownership irrelevant
7	Y	Update intent file without matching module refs	NO-OP_PROMOTION_INTENT Ownership irrelevant
8	Y	Update intent and matching module refs	PROMOTION_INTENT

Rollback any failed apply-time governance tests that have already merged to main. This discomfort is intentional and will be addressed next by shifting governance left.

Shift Governance Left

Up to now, governance has been enforced at apply time. This allowed you to observe governance failures clearly, but it also allowed bad changes to reach main before failing. Mature delivery models run the same checks during pull request validation, preventing malformed or unauthorised changes from reaching main.

From this point forward, governance moves earlier in the lifecycle. The rules do not change; only where they run changes.

25. Update the three **pr-plan** workflows to enable ownership logic by changing **FEATURE_PHASE** from 1 to **3**:

```
.github\workflows\pr-plan-dev.yml  
.github\workflows\pr-plan-test.yml  
.github\workflows\pr-plan-prod.yml
```

Important: The PR plan workflows should be required-check compatible: they should trigger for pull requests to main and return success even when the changed path is not relevant to that environment. This avoids path-filtered workflows remaining pending when made required.

26. Commit and merge the plan workflow updates:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b plan-updates
git add .github\workflows\pr-plan-dev.yml
git add .github\workflows\pr-plan-test.yml
git add .github\workflows\pr-plan-prod.yml
git commit -m "Update plan workflows with governance checks"
git push -u origin plan-updates
```

27. Create and merge the pull request.

Make PR Governance Checks Required

28. In GitHub, open the env-roots repository.

29. Navigate to Settings > Branches.

30. Create or edit the branch protection rule for **main**.

31. Enable **Require status checks to pass before merging**.

32. Select the PR plan workflow checks. Depending on the workflow names emitted by GitHub, these may appear as pr-plan-dev, pr-plan-test, and pr-plan-prod, or as job names within those workflows.

33. Save the branch protection rule. The goal is that PR governance failures prevent merging to main.

Test Governance Before Hitting Main

Attempt an unauthorised environment update in **TEST**. This should fail during PR validation and should not merge to main.

34. In **envs/test/terraform.tfvars**, change a safe environment-level setting, such as a subnet CIDR value.

35. Push the change:

```
cd c:\pipeline-repos\env-roots
git checkout main
```

```
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b test-env-change
git add envs\test\terraform.tfvars
git commit -m "Update TEST environment"
git push -u origin test-env-change
```

36. Create the pull request.

37. Observe the governance-integrated PR plan workflow fail.

38. Attempt to merge the pull request. The merge should be blocked by the required status check.

39. Close the pull request and clean up the branch:

```
git switch main
git push origin --delete test-env-change
git branch -D test-env-change
```

40. Now attempt a similar PROD environment update after ensuring your GitHub username is listed as a prod owner. The PR plan should pass governance checks. After merge, **change-prod** should also run the same governance checks before applying. Roll back the change once tested.

Key Takeaway

- Local environment changes can safely use the commit itself as intent when the blast radius is low
- The explicitness of intent must scale with blast radius
- Cross-environment or shared-module promotions require explicit, scoped declarations
- Environment ownership governs all changes to that environment, not just module upgrades
- Governance should run before merge as well as before apply

At this point, module promotion is governed and environment configuration changes are governed. This raises one final question: Who governs changes to the governance itself?

Phase 3. Governance-of-Governance

This part introduces and enforces a governance-of-governance workflow, allowing you to observe how existing governance state affects enforcement, including bootstrap failure and break-glass recovery.

Introduce the Governance Workflow

1. Copy the supplied governance workflow into env-roots:

From: c:\qatfadvaws\artifacts\governance\governance.yml

To: c:\pipeline-repos\env-roots\.github\workflows\governance.yml

2. Commit and merge the workflow:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b gov-workflow
git add .github\workflows\governance.yml
git commit -m "Add governance workflow"
git push -u origin gov-workflow
```

3. Create and merge the pull request. At this point, the workflow exists, but nothing is enforcing it. Do not run it manually.
4. Open **governance/owners.yml** on main and confirm the governance owner list does not include your username.
5. At this moment, governance is defined, governance protection exists, but governance is not yet enforced.

Enforce Governance-of-Governance

6. In GitHub, open Settings > Branches.
7. Edit the branch protection rule for main.
8. Add the governance workflow status check as a **required** status check.
9. Save the branch protection rule. This is the exact moment the system becomes self-protecting.

Attempt Normal Governance Changes - Expected Failure

10. Modify **governance/owners.yml** and attempt to add a new dev owner.
11. Push the change:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b gov-test-1
git add governance\owners.yml
git commit -m "Governance test 1"
```

```
git push -u origin gov-test-1
```

12. Open a pull request to main.

Expected result: the governance workflow runs and fails because no authorised governance owner exists.

13. Close the pull request and clean up the branch:

```
git switch main
git push origin --delete gov-test-1
git branch -D gov-test-1
```

14. Now add yourself as a governance owner in **governance/owners.yml** and try again using a new branch.

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b gov-test-2
git add governance\owners.yml
git commit -m "Governance test 2"
git push -u origin gov-test-2
```

15. Open the pull request. Expected result: the **same failure** occurs. You cannot grant yourself authority through a governed process when there are currently no authorised governance owners.

16. Close the pull request and clean up the branch:

```
git switch main
git push origin --delete gov-test-2
git branch -D gov-test-2
```

Break-Glass Recovery

Every real governance system needs break-glass recovery. The danger is pretending that it does not.

17. In GitHub, temporarily allow your account to bypass the main branch protection rule, or temporarily relax the required governance status check.

18. Edit **governance/owners.yml** directly on main and add your GitHub username as a governance owner.

19. Commit the change **directly** to main.

20. Immediately restore the branch protection rule or bypass settings.

The break-glass action should be rare, visible, and reversible. In a production environment, this would normally require separate emergency approval and review.

Normal Governance Operation

21. Synchronize your local main branch:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
```

22. Retry a governance change, such as adding a dev owner to governance/owners.yml.

23. Create a pull request. Expected result: the governance workflow passes because your GitHub username is now an authorised governance owner. The PR can be merged normally.

Key Learning Points

- Governance workflows must exist before enforcement can be enabled
- Enabling enforcement can expose historical governance debt
- Empty governance is a valid state, but not a safe one
- Break-glass access is mandatory, not optional
- Governance itself must be governed

***** Congratulations, you have completed this lab *****

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026